PHASE 18 OF 18

HTTP Client, Networking & Internationalization

Networking basics, URI/URL and encoding, TCP sockets, the Java 11 HttpClient (sync, async, JSON, timeouts, retries), a local HTTP server for API mocks, and i18n with Locale, NumberFormat (₹ and lakh grouping), localized dates, MessageFormat, ResourceBundle and Collator

IN THIS PHASE

18.1 Networking Basics
18.2 TCP Sockets (java.net)
18.3 The Java 11 HttpClient (java.net.http)
18.4 A Local HTTP Server for Mocks and Tests
18.5 Internationalization (i18n) and Localization (l10n)
18.6 Interview Questions
18.7 Quick Revision

18.1 Networking Basics

Term	Meaning
IP address	Identifies a machine on a network (`192.168.1.10`, IPv6 `::1`); `localhost` = `127.0.0.1`
Port	Identifies a process on that machine (0–65535): HTTP 80, HTTPS 443, Appium 4723, PostgreSQL 5432
Socket	One endpoint of a two-way connection: IP + port
TCP	Connection-oriented, reliable, ordered (HTTP, DB connections)
UDP	Connectionless, fast, no delivery guarantee (DNS, streaming)
HTTP / HTTPS	Request–response protocol over TCP (HTTPS = HTTP + TLS encryption)

URL anatomy, URI vs URL

```
https://api.bank.example:8443/v1/accounts/SB-1001?type=savings&page=2#top
\___/   \_____________/ \__/\___________________/\_________________/\__/
scheme        host       port        path               query      fragment
```

```
URI u = URI.create("https://api.bank.example:8443/v1/accounts/SB-1001?type=savings&page=2#top");
u.getScheme();   // https          u.getHost();   // api.bank.example
u.getPort();     // 8443           u.getPath();   // /v1/accounts/SB-1001
u.getQuery();    // type=savings&page=2            u.getFragment(); // top
URI.create("https://a.com/x/y/").resolve("../z?q=1");   // https://a.com/x/z?q=1
URL url = u.toURL();              // URL constructors are deprecated (Java 20): go via URI
```

A **URI** identifies a resource (syntax only, no network access). A **URL** is a URI that also says how to locate it. In modern Java, build a `URI`, and convert with `toURL()` only for legacy APIs.

URL encoding

Query parameters must be **percent-encoded**: spaces, `&`, `=`, `/` and non-ASCII characters would otherwise break the URL.

```
String q = URLEncoder.encode("name=Rohit Kumar&city=Pune/MH", StandardCharsets.UTF_8);
// name%3DRohit+Kumar%26city%3DPune%2FMH
URLDecoder.decode("Rohit+Kumar%26co", StandardCharsets.UTF_8);    // Rohit Kumar&co

InetAddress addr = InetAddress.getByName("localhost");            // DNS lookup
System.out.println(addr.getHostAddress());                        // 127.0.0.1
```

18.2 TCP Sockets (java.net)

Raw sockets are the foundation under HTTP, JDBC drivers and messaging clients. You'll rarely write them yourself, but understanding them helps with debugging (timeouts, connection refused, ports).

```
// SERVER: listens on a port, echoes one line back in upper case
try (ServerSocket server = new ServerSocket(5050)) {
    try (Socket client = server.accept();                // blocks until a client connects
         BufferedReader in = new BufferedReader(
                 new InputStreamReader(client.getInputStream(), UTF_8));
         PrintWriter out = new PrintWriter(client.getOutputStream(), true, UTF_8)) {
        out.println("ECHO: " + in.readLine().toUpperCase());
    }
}

// CLIENT
try (Socket socket = new Socket()) {
    socket.connect(new InetSocketAddress("localhost", 5050), 2000);    // connect timeout 2 s
    socket.setSoTimeout(3000);                                          // read timeout 3 s
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true, UTF_8);
    BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream(), UTF_8));
    out.println("balance enquiry");
    System.out.println(in.readLine());                                  // ECHO: BALANCE ENQUIRY
}
```

Exception	Usual meaning
`ConnectException: Connection refused`	Nothing is listening on that host and port (service down, wrong port)
`SocketTimeoutException`	Connect or read took longer than the timeout
`UnknownHostException`	DNS couldn't resolve the host name
`SSLHandshakeException`	TLS problem: untrusted certificate, wrong protocol, missing CA in the truststore
`BindException: Address already in use`	The server port is already taken

> **TIP**
>
> **Always set timeouts.** By default, a socket read can block forever, and a hung downstream service then freezes your threads. This is one of the most common production incidents.

18.3 The Java 11 HttpClient (java.net.http)

The modern, built-in HTTP client replaces the old `HttpURLConnection`. It supports HTTP/1.1 and **HTTP/2**, **synchronous and asynchronous** calls (`CompletableFuture`), builders, timeouts, redirects, authentication, and WebSockets.

```
HttpClient (reusable, thread-safe: create ONCE)
   +- send(request, bodyHandler)       -> HttpResponse<T>                    (blocking)
   +- sendAsync(request, bodyHandler)  -> CompletableFuture<HttpResponse<T>>  (async)
```

Creating the client

```
HttpClient client = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_2)              // falls back to HTTP/1.1 if needed
        .connectTimeout(Duration.ofSeconds(5))
        .followRedirects(HttpClient.Redirect.NORMAL)     // follow 3xx (except HTTPS -> HTTP)
        .build();                                        // reuse this instance everywhere
```

GET with query parameters and headers

```
String accountId = "SB-1001";
String type = URLEncoder.encode("savings & current", StandardCharsets.UTF_8);
HttpRequest get = HttpRequest.newBuilder()
        .uri(URI.create(BASE + "/v1/accounts/" + accountId + "?type=" + type))
        .header("Accept", "application/json")
        .header("Authorization", "Bearer " + token)      // never log this header!
        .timeout(Duration.ofSeconds(10))                 // per-request response timeout
        .GET()                                           // the default method
        .build();

HttpResponse<String> res = client.send(get, HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());                    // 200
System.out.println(res.headers().firstValue("content-type").orElse("?"));
System.out.println(res.body());                          // {"id":"SB-1001","balance":2500.50}
```

POST, PUT and DELETE with a JSON body

```
String json = """
        {"from":"SB-1001","to":"SB-2002","amount":1500.00,"currency":"INR"}
        """;
HttpRequest post = HttpRequest.newBuilder()
        .uri(URI.create(BASE + "/v1/transfers"))
        .header("Content-Type", "application/json")
        .header("Idempotency-Key", UUID.randomUUID().toString())   // safe retries for payments
        .POST(HttpRequest.BodyPublishers.ofString(json))
        .build();
HttpResponse<String> created = client.send(post, HttpResponse.BodyHandlers.ofString());
if (created.statusCode() != 201) {
    throw new IllegalStateException(
            "Transfer failed: " + created.statusCode() + " " + created.body());
}

HttpRequest put = HttpRequest.newBuilder(URI.create(BASE + "/v1/accounts/SB-1001/nickname"))
        .PUT(HttpRequest.BodyPublishers.ofString("{\"nickname\":\"Salary\"}"))
        .header("Content-Type", "application/json").build();
HttpRequest del = HttpRequest.newBuilder(URI.create(BASE + "/v1/beneficiaries/77"))
        .DELETE().build();
HttpRequest patch = HttpRequest.newBuilder(URI.create(BASE + "/v1/cards/9"))
        .method("PATCH", HttpRequest.BodyPublishers.ofString("{\"blocked\":true}")).build();
```

`BodyPublishers` **(request body)**	`BodyHandlers` **(response body)**
`ofString(s)` / `ofString(s, charset)`	`ofString()`: the whole body as a String
`ofFile(path)`	`ofFile(path)`: stream to a file (downloads)
`ofByteArray(bytes)`	`ofByteArray()`, `ofInputStream()`
`ofInputStream(supplier)`	`ofLines()`: `Stream<String>` (large text responses)
`noBody()`	`discarding()`: ignore the body

> **NOTE**
> `HttpClient` does **not** throw for 4xx or 5xx statuses. A 404 or 500 is a normal `HttpResponse`, so **always check** `statusCode()`. Exceptions are thrown only for transport problems: `HttpConnectTimeoutException`, `HttpTimeoutException`, `ConnectException`, `IOException`, and `InterruptedException` for `send`.

Asynchronous and parallel calls

```
CompletableFuture<String> balance = client
        .sendAsync(get, HttpResponse.BodyHandlers.ofString())
        .thenApply(HttpResponse::body);                          // no thread blocked meanwhile

List<URI> uris = List.of(URI.create(BASE + "/v1/accounts/SB-1"),
                         URI.create(BASE + "/v1/accounts/SB-2"),
                         URI.create(BASE + "/v1/accounts/SB-3"));
List<CompletableFuture<String>> calls = uris.stream()
        .map(u -> client.sendAsync(HttpRequest.newBuilder(u).build(),
                                   HttpResponse.BodyHandlers.ofString())
                        .thenApply(HttpResponse::body))
        .toList();
CompletableFuture.allOf(calls.toArray(CompletableFuture[]::new)).join();   // all 3 in parallel
calls.forEach(c -> System.out.println(c.join()));
```

A simple retry with backoff (for idempotent or keyed requests)

```
static HttpResponse<String> sendWithRetry(HttpClient client, HttpRequest req, int maxAttempts)
        throws IOException, InterruptedException {
    IOException last = null;
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            HttpResponse<String> r = client.send(req, HttpResponse.BodyHandlers.ofString());
            if (r.statusCode() < 500 && r.statusCode() != 429) return r;   // done
        } catch (IOException e) {
            last = e;                                            // network glitch: retry
        }
        Thread.sleep(200L * (1L << (attempt - 1)));                          // 200, 400, 800 ms
    }
    throw new IOException("Gave up after " + maxAttempts + " attempts", last);
}
```

> **PITFALL**
> Only retry **idempotent** operations (GET, PUT, DELETE), or POSTs protected by an **idempotency key**. Blindly retrying a payment POST after a timeout can debit the customer twice.

JSON mapping

The JDK has no JSON parser. In practice, combine `HttpClient` with **Jackson** (or Gson) and records:

```
record AccountDto(String id, BigDecimal balance) { }
ObjectMapper mapper = new ObjectMapper();                        // com.fasterxml.jackson
AccountDto acc = mapper.readValue(res.body(), AccountDto.class);
String body = mapper.writeValueAsString(new AccountDto("SB-9", new BigDecimal("10.00")));
```

> **TIP**
> For API **test suites**, REST Assured (built on the same ideas) gives fluent given/when/then assertions. The plain `HttpClient` is ideal for utilities, health checks, test-data setup and Appium-adjacent backend calls without extra dependencies.

HttpClient vs HttpURLConnection

`HttpURLConnection` **(Java 1.1)**	`HttpClient` **(Java 11)**
Clunky, mutable API; manual streams	Fluent builders; immutable requests
HTTP/1.1 only	HTTP/2 and HTTP/1.1, WebSockets
Blocking only	Synchronous **and** asynchronous (`CompletableFuture`)
Easy to leak connections	Connection pooling built in

18.4 A Local HTTP Server for Mocks and Tests

The JDK ships a lightweight server (`com.sun.net.httpserver`, module `jdk.httpserver`). It's handy for **stubbing a downstream API** in tests or demos. For full-featured mocking, teams use WireMock or MockWebServer.

```
HttpServer server = HttpServer.create(new InetSocketAddress(8089), 0);
server.createContext("/v1/accounts/SB-1001", exchange -> {
    byte[] body = "{\"id\":\"SB-1001\",\"balance\":2500.50}".getBytes(UTF_8);
    exchange.getResponseHeaders().add("Content-Type", "application/json");
    exchange.sendResponseHeaders(200, body.length);
    try (OutputStream os = exchange.getResponseBody()) { os.write(body); }
});
server.createContext("/v1/transfers", exchange -> {
    String req = new String(exchange.getRequestBody().readAllBytes(), UTF_8);
    int status = req.contains("\"amount\":0") ? 422 : 201;
    exchange.sendResponseHeaders(status, -1);                    // -1 = no body
    exchange.close();
});
server.setExecutor(Executors.newVirtualThreadPerTaskExecutor()); // Java 21
server.start();
// ... run client calls against http://localhost:8089 ...
server.stop(0);
```

```
> jwebserver -p 8000 -d C:\reports      (Java 18+) serve static files, e.g. test reports
```

18.5 Internationalization (i18n) and Localization (l10n)

i18n means designing software so that it can adapt to languages and regions. **l10n** means providing the actual translations and formats for a locale. In banking apps this shows up in currency formats, dates, number separators and multi-language messages (English, Hindi, Marathi...).

Locale

```
Locale enIN = Locale.of("en", "IN");             // Java 19+ (new Locale(...) is deprecated)
Locale hiIN = Locale.of("hi", "IN");
Locale tag  = Locale.forLanguageTag("mr-IN");    // BCP 47 tag: Marathi (India)
Locale.US;  Locale.GERMANY;  Locale.JAPAN;       // constants
Locale.getDefault();                             // JVM default (OS or -Duser.language)
enIN.getDisplayCountry();                        // India
hiIN.getDisplayLanguage(hiIN);                   // [Hindi] in Devanagari script
```

PITFALL

Calls without a locale (`toUpperCase()`, `String.format`, `NumberFormat.getInstance()`) use the **JVM default**, which differs between your laptop, the CI server and production. The classic bug: `"TITLE".toLowerCase()` in a Turkish locale yields a dotless `ı`. Pass `Locale.ROOT` for machine-readable text, and an explicit locale for user-facing text.

Numbers, currency and percentages (verified on JDK 21)

```
double amt = 1234567.891;
NumberFormat.getCurrencyInstance(Locale.of("en", "IN")).format(amt);  // ₹1,234,567.89
NumberFormat.getCurrencyInstance(Locale.US).format(amt);              // $1,234,567.89
NumberFormat.getCurrencyInstance(Locale.GERMANY).format(amt);         // 1.234.567,89 €
NumberFormat.getCurrencyInstance(Locale.JAPAN).format(amt);           // ¥1,234,568 (no decimals)
NumberFormat.getNumberInstance(Locale.GERMANY).format(amt);           // 1.234.567,891

NumberFormat pct = NumberFormat.getPercentInstance(Locale.of("en", "IN"));
pct.setMinimumFractionDigits(2);
pct.format(0.0725);                                       // 7.25% (default rounds: 7%)

NumberFormat compact = NumberFormat.getCompactNumberInstance(
        Locale.of("en", "IN"), NumberFormat.Style.SHORT);             // Java 12
compact.setMaximumFractionDigits(1);
compact.format(150_000);                                              // 1.5L
compact.format(12_500_000);                                           // 1.2Cr

Currency inr = Currency.getInstance(Locale.of("en", "IN"));
inr.getCurrencyCode();  inr.getDefaultFractionDigits();               // INR, 2

NumberFormat.getNumberInstance(Locale.GERMANY).parse("1.234,5");    // 1234.5 (parsing too)
```

PITFALL

A common assumption is that the `en-IN` locale, or the pattern `#,##,##0.00`, produces Indian grouping (`12,34,567.89`). On the standard JDK it **doesn't**: `DecimalFormat` supports only one grouping size, so both give `1,234,567.89`. Use ICU4J (`com.ibm.icu.text.NumberFormat`), or a small helper like the one below, when statements must show lakh/crore grouping.

```
static String indianFormat(BigDecimal amount) {             // 1234567.891 -> ₹12,34,567.89
    String plain = amount.setScale(2, RoundingMode.HALF_UP).abs().toPlainString();
    String intPart = plain.substring(0, plain.indexOf('.'));
    String decPart = plain.substring(plain.indexOf('.'));
    String last3 = intPart.length() > 3 ? intPart.substring(intPart.length() - 3) : intPart;
    String rest  = intPart.length() > 3 ? intPart.substring(0, intPart.length() - 3) : "";
    rest = rest.replaceAll("\\B(?=(\\d{2})+$)", ",");   // pairs before last three
    String sign = amount.signum() < 0 ? "-" : "";
    return sign + "₹" + (rest.isEmpty() ? "" : rest + ",") + last3 + decPart;
}
indianFormat(new BigDecimal("1234567.891"));   // ₹12,34,567.89
indianFormat(new BigDecimal("950.5"));         // ₹950.50
```

Localized dates

```
LocalDate d = LocalDate.of(2026, 8, 15);
DateTimeFormatter longFmt = DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG);
d.format(longFmt.withLocale(Locale.US));                      // August 15, 2026
d.format(longFmt.withLocale(Locale.of("en", "IN")));          // 15 August 2026
d.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM)
        .withLocale(Locale.GERMANY));                         // 15.08.2026
d.format(DateTimeFormatter.ofPattern("EEEE, d MMMM yyyy", Locale.of("hi", "IN")));
                                      // [Shanivaar, 15 Agast 2026] in Devanagari
```

ResourceBundle: externalised, translated messages

NOTE

The PDF font can't draw Devanagari, so Hindi and Marathi examples are shown **transliterated in [brackets]**. In your `.properties` files, IDE and console they appear in the real script (verified on JDK 21 with UTF-8 output).

```
src/main/resources/
  messages.properties          greeting=Welcome, {0}!           (default / fallback)
  messages_hi.properties       greeting=[Swagat hai], {0}!
  messages_mr.properties       greeting=[Swagat aahe], {0}!
```

```
ResourceBundle rb = ResourceBundle.getBundle("messages", Locale.of("hi", "IN"));
String text = MessageFormat.format(rb.getString("greeting"), "Rohit");   // [Swagat hai], Rohit!
// Lookup order: messages_hi_IN -> messages_hi -> messages_<default locale> -> messages
```

> **NOTE**
> Since Java 9, `.properties` resource bundles are read as **UTF-8**, so Hindi and Marathi text can be written directly (no more `\u0938` escapes). Spring's `MessageSource` builds on the same idea.

MessageFormat: placeholders with formatting

```
String msg = MessageFormat.format(
        "Dear {0}, your account {1} was debited with {2,number,#,##0.00} on {3}.",
        "Rohit", "SB-1001", 2500.5, "15-08-2026");
// Dear Rohit, your account SB-1001 was debited with 2,500.50 on 15-08-2026.
```

> **PITFALL**
> In `MessageFormat`, a single quote starts a quoted (literal) section. `MessageFormat.format("Don't {0}", "x")` prints `Dont {0}`: the quote disappears **and** the placeholder isn't replaced. Write `"Don''t {0}"` to get `Don't x`.

Collator: locale-aware sorting

```
List<String> words = new ArrayList<>(List.of("zebra", "Apple", "éclair", "banana"));
Collections.sort(words);                              // [Apple, banana, zebra, éclair]
                                                      //   (by code point)
words.sort(Collator.getInstance(Locale.ENGLISH));  // [Apple, banana, éclair, zebra]
                                                      //   (human order)
```

18.6 Interview Questions

Q. What is the difference between TCP and UDP?

TCP is connection-oriented, reliable and ordered (HTTP, databases). UDP is connectionless and faster but has no delivery or order guarantees (DNS, media streaming).

Q. What is the difference between URI and URL in Java?

A URI is a syntactic identifier (parsing, resolving, encoding) with no network access. A URL can be used to open connections. Build URIs (`URI.create`) and convert with `toURL()` when needed; URL constructors are deprecated since Java 20.

Q. How do you make an HTTP call in modern Java?

Create one reusable `HttpClient` (with a connect timeout), build an `HttpRequest` (URI, headers, method, body publisher, timeout), then call `send` with a `BodyHandler` for a blocking call, or `sendAsync` to get a `CompletableFuture`.

Q. Does HttpClient throw an exception for a 404 or 500 response?

No. HTTP error statuses are normal responses, so check `statusCode()`. Exceptions only signal transport failures such as timeouts, refused connections or I/O errors.

Q. Why must you set timeouts, and which ones exist?

Without them, calls to a slow or hung service can block threads indefinitely. `HttpClient` has `connectTimeout` (on the builder) and a per-request `timeout`; sockets have a connect timeout and `setSoTimeout` (read timeout).

Q. When is it safe to retry an HTTP request?

For idempotent methods (GET, PUT, DELETE), or non-idempotent ones like payment POSTs protected by an idempotency key. Use limited attempts with exponential backoff, and don't retry 4xx client errors (except 429).

Q. What is the difference between i18n and l10n?

Internationalization designs software to support many locales (externalised strings, locale-aware formatting). Localization supplies the translations and regional formats for a specific locale.

Q. How do you format currency for a locale?

Use `NumberFormat.getCurrencyInstance(locale)`, or `BigDecimal` with an explicit scale for calculations. Note that the standard JDK doesn't apply Indian lakh grouping; use ICU4J or a custom formatter.

Q. What is a ResourceBundle?

A locale-specific set of key–value messages (usually `.properties` files such as `messages_hi.properties`), resolved with a fallback chain from the most specific to the default bundle.

Q. Why is relying on the default locale risky?

Output changes with machine settings, so formats, case conversion and parsing can break in CI or production. Pass explicit locales (`Locale.ROOT` for machine text).

18.7 Quick Revision

- TCP = reliable connection; UDP = fast datagrams. A socket = IP + port. Always set **connect and read timeouts**.
- Build `URI`s (not `new URL`); encode query values with `URLEncoder.encode(v, UTF_8)`.
- Socket errors: refused (nothing listening), timeout, unknown host, SSL handshake.
- `HttpClient`: create once (thread-safe), HTTP/2, `connectTimeout`; `HttpRequest.newBuilder().uri().header().timeout().GET()/POST(BodyPublishers.ofString(json))`.
- `send` (blocking) vs `sendAsync` (CompletableFuture); `BodyHandlers.ofString`/`ofFile`/`ofLines`.
- **4xx/5xx don't throw**: check `statusCode()`. Retry only idempotent or keyed requests, with backoff.
- JSON: Jackson + records. Local mocks: `com.sun.net.httpserver.HttpServer`; `jwebserver` for static files.
- Locale: `Locale.of("en","IN")`, `forLanguageTag`; avoid hidden default-locale behaviour.
- `NumberFormat`: currency, percent, compact (`1.5L`, `1.2Cr`); the JDK has **no lakh grouping**: use ICU4J or a helper.
- Dates: `DateTimeFormatter.ofLocalizedDate(FormatStyle.X).withLocale(l)`.
- `ResourceBundle` + `MessageFormat` for translated messages (UTF-8 `.properties` since Java 9); `Collator` for human sorting.